

ROOTING FOR KEPLER

*a numerical benchmark of
Kepler solvers and starting guesses*

An empirical study of classical, domain-specialised, and 2024–2025 with-memory Newton-type variants applied to Kepler’s equation, evaluated on RadVel’s radial-velocity fitting pipeline.

125,800 instrumented solves over a 7,400-point (e, M) grid, with iterative solvers checked against published convergence orders before entering the benchmark.

SUBSECTION A2 | STUDENT IDs

2105036 2105050 2105053 2105056 2105059

SUMMARY OF FINDINGS

The project compares recent with-memory root finders with established Kepler solvers, varying both the starting guess and the orbital regime. Higher formal order reduces the number of iterations, but does not by itself imply a cheaper or more reliable solve.

1. **The verification gates pass.** On eight test functions from the NWM11 paper, measured orders agree with the published values within 1%; the two with-memory base error constants also match their respective papers. NWM9 has not yet been tested on its own paper's example functions.
2. **Fewer iterations do not ensure lower cost.** With selected starting guesses, NWM11 needs a median of one iteration against Danby's two. Its modelled median cost is 6.34 versus 7.68, but its *mean cost among converged solves* is 9.55 versus 7.48.
3. **The timed Python implementations favour simpler methods.** On the timed subset, the median solve takes $5.25\ \mu\text{s}$ for NWM11 with its canonical start and $4.46\ \mu\text{s}$ for Danby with Napier's start. These are implementation timings, not a compiled-algorithm comparison.
4. **Reliability depends on the starting guess.** Across all four guesses, NWM9 and NWM11 fail on 4.27% and 4.86% of solves in the defined hard corner, versus Danby's 0.13%. NWM9 with Napier's start has no failures on this grid.
5. **The proposed fused-trigonometry benefit remains untested.** This Python code evaluates sin and cos separately. Under one-pair-per-call accounting, the representative NWM11 and Danby solves both cost five calls; a compiled speed advantage cannot be inferred from these data.
6. **Downstream shifts are small in the tested injection.** Fitted-parameter shifts from varying solver tolerance are at least about 10^3 times smaller than the Monte Carlo spread for this moderately eccentric injected orbit.

PRACTICAL READING OF THE BENCHMARK

Markley is fastest on the timed grid subset and has no failures there; its closed-form computation is not fully captured by the trigonometric cost proxy. Danby remains a strong iterative baseline. The with-memory schemes reduce iteration counts, but this experiment does not establish a total-cost or wall-clock advantage over Danby.

1 WHAT WAS BUILT

The deliverable is a cost-instrumented benchmark harness (`keplerbench`) with a shared solver interface and configured experiments. The results and figures were regenerated using the commands below; the numerical tables in this document were then checked against those outputs.

1.1 ARCHITECTURE

All five solvers and four starting guesses expose the same calling interface. The four iterative solvers use a residual stopping test; Markley is a closed-form method that ignores the requested tolerance and starting guess. Solver evaluations pass through a `KeplerProblem` object that records what each call needed — how many distinct points were touched, how many required a full (sin, cos) pair against sin or cos alone, and how many derivatives were synthesised from interpolation rather than evaluated. That instrumentation is what makes Section 4 possible.

Reference roots come from an independent 50-digit bracketing solve in `mpmath`, never from one of the methods under test.

1.2 REPRODUCIBILITY

The benchmark and verification raw tables have provenance sidecars with their configuration and source commit. Several derived and downstream CSVs do not yet have sidecars, so the blanket provenance claim would be incorrect. The results used here were regenerated from implementation commit 48dcad1 with Python 3.12.6; the configured cost weights were calibrated separately on Python 3.14.7.

Command	Produces
<code>scripts/run_verification.py</code>	<code>results/verification/</code> — order and correctness gate
<code>scripts/run_grid_benchmark.py</code>	<code>results/grid_benchmark/</code> — 125,800 solves
<code>scripts/run_error_propagation.py</code>	<code>results/error_propagation/</code> — tolerance sweep
<code>scripts/run_monte_carlo.py</code>	<code>results/monte_carlo/</code> — noise realisations
<code>scripts/make_report_figures.py</code>	<code>figures/</code> — every figure below

2 CANDIDATE METHODS AS IMPLEMENTED

Category	Method	Order	Role in the benchmark
Baseline	Newton–Raphson	2	Textbook anchor; one evaluation point, one $(\sin, \cos)$ pair per iteration.
Kepler solver	Danby’s method	4	RadVel’s production solver; domain-tuned to this equation.
	Markley’s method	—	Closed-form solver; no iteration, constant cost.
With-memory	NWM9 (2024)	8.8989	Single self-accelerating parameter in the <i>third</i> step of an optimal eighth-order base.
	NWM11 (2025)	10.7446	Bi-parametric; 3 evaluation points, 1 full $(\sin, \cos)$ pair per iteration.
Guess layer	Napier’s guess (2024)	n/a	Symbolic-regression start, run in front of every iterative solver.

THREE CORRECTIONS TO THE PROPOSAL’S METHOD TABLE

1. The with-memory schemes are **not derivative-free**. Both use $\varphi'(s_k)$ explicitly in their first sub-step. For Kepler this is favourable rather than otherwise: f and f' come from a single $(\sin, \cos)$ pair, which is exactly why the “3 points, 1 pair” economy is achievable at all.
2. NWM9 is **not the controlled sibling of NWM11**. They come from two different papers with two different eighth-order base methods and the parameter placed in different sub-steps. NWM11’s true same-base control is **NWM10** (R-order 10), defined in the same paper as NWM11. This matters for interpreting Section 6, where the two behave qualitatively differently.
3. The phrase “at no additional evaluation points per iteration” is misleading. Both schemes use three evaluation points per iteration against Newton’s one. What costs nothing extra is the *derivative information*, obtained by Hermite interpolation over

points already paid for.

3 VERIFICATION: THE GATE BEFORE THE BENCHMARK

The iterative solvers were checked for convergence order before the grid benchmark. This reduces the risk of transcription errors, but is not an independent proof of each implementation.

3.1 MEASURED CONVERGENCE ORDER

Each solver was run on the eight nonlinear test functions of the NWM11 paper, from the paper's own starting points, at 2000-digit working precision. NWM9 comes from a different paper with its own examples, but it was gated on the same NWM11 set; checking it on its own paper's functions remains to be done (experiments/verification.py).

Solver	Claimed	Measured (mean)	Range over 8 functions	Gate
Newton	2.0000	2.0000	2.0000 – 2.0000	PASS
Danby	4.0000	4.0000	4.0000 – 4.0000	PASS
NWM9	8.8989	8.9114	8.8746 – 8.9816	PASS
NWM11	10.7446	10.7605	10.7147 – 10.8336	PASS

Every measurement sits slightly *above* the claim, which is correct: both papers state their R-order as a lower bound (“at least”).

WHY 2000 DIGITS, AND NOT 16

An order- p method multiplies its correct digits by p each step. From an error of 10^{-1} , an order-9 method passes 10^{-9} , 10^{-81} , 10^{-729} . The estimator needs three consecutive usable errors to form one estimate, so the working precision must outrun p^3 digits or the sequence hits the floor before a single estimate exists. At 100 digits NWM9 measured anywhere in 8.87–10.19; at 2000 it settles to 8.87–8.98, and 3000 buys nothing further. Double precision cannot measure these methods at all.

3.2 THE ORDER ALONE IS NOT SUFFICIENT EVIDENCE

Both papers' iteration formulas contain nested fractions whose PDF text layer admits several readings, and for both schemes *more than one reading yields the correct order*. Measuring order 8 would therefore not have detected a mis-transcription.

What does detect it is each paper's explicit error *constant*. Each scheme was driven for exactly one step from a point 10^{-24} from the root and its eighth-order constant compared against the published closed form:

Scheme	Published constant	Agreement (relative)
NWM9	Eq. (7): $c_2^2 K[(\alpha + c_2)K + c_4]$, $K = c_2 + 5c_2^2 - c_3$	$\leq 1.8 \times 10^{-22}$
NWM11	Eq. (2.6): $(\alpha + c_2)^2 c_3 (c_2 c_3 - c_4)$	$\leq 3.5 \times 10^{-22}$

A wrong grouping in NWM11's $w'(t_k)$ missed this constant by roughly 60 orders of magnitude while still converging at order 8. Three of the eight test functions also had to be corrected: superscripts detach in the PDF text layer, so e^{s^2} extracts as e^s followed by a stray 2. These were caught because the published roots did not zero the mis-transcribed functions.

3.3 TWO-STAGE VERIFICATION

Setting each scheme’s self-accelerating parameters to zero must recover the underlying memoryless method. Both do, measuring exactly 8.0000 on all eight functions, which attributes the lift to 8.8989 and 10.7446 to the memory rather than to a coincidence in the base method.

3.4 CORRECTNESS: THE RIGHT ROOT, NOT JUST A ROOT

A solver that jumps to a different branch of Kepler’s equation reports a tiny residual and a converged flag while every downstream quantity is wrong. Over 4,975 (e, M) points spanning the full range:

Solver	Converged	Wrong root	Max $ E - E_{\text{ref}} $
Newton	99.54%	0	3.38×10^{-10}
Danby	99.72%	0	1.65×10^{-10}
NWM9	98.25%	0	2.32×10^{-10}
NWM11	99.56%	0	2.23×10^{-10}

No method converged to a wrong branch anywhere. Non-convergence is reported honestly and is treated as a robustness result (Section 6), not as a correctness failure.

4 KEPLER-SPECIFIC COST ACCOUNTING

4.1 MEASURED PER-ITERATION COST

Counted from the instrumented code rather than transcribed from the papers:

Method	Eval. points	(sin, cos) pairs	sin only	Synth. derivatives	Order
Newton	1	1	1	0	2.0000
Danby	1	1	1	0	4.0000
Markley	2	1	1	0	—
NWM9	3	1	3	3	8.8989
NWM11	3	1	3	3	10.7446

In the measured steady-state iteration, both with-memory schemes use three evaluation points but only one full (sin, cos) pair. Three derivatives are synthesised by interpolation; the first iteration has not yet accumulated this information.

One count in this table is an artefact of the harness, not of the methods. The shared iteration loop calls $f(E)$ itself after each step to compute the residual for the stopping test. That inflates the sin-only column by exactly one per iteration for *every* solver — which is why Newton and Danby show a sin-only cost of 1 despite needing nothing beyond their single (sin, cos) pair. The with-memory schemes’ own sin-only cost is therefore 2, not 3. The inflation is uniform, so it does not distort comparisons, but it must be subtracted before any of these figures is quoted against a paper’s table.

4.2 COSTING ASSUMPTIONS AND THEIR LIMITS

The proposal’s argument rests on a specific empirical claim: that one sincos call returns both $\sin E$ and $\cos E$ “at roughly the cost of one transcendental evaluation”. The benchmark uses

the following configured weights, calibrated on a separate CPython 3.14.7 run and normalised to one sin call:

Operation	Relative cost	Proposal assumed
sin only	1.000	1
cos only	1.04	1
(sin, cos) pair	2.34	≈ 1
Synthesised derivative (Hermite)	1.76	—

These are the code’s default weights; for example, Newton’s modelled steady-state cost is $2.34 + 1 = 3.34$. The current Python 3.12.6 environment did not reproduce a stable derivative weight: the default calibration raised an error when its baseline subtraction gave a non-positive sin cost, and a longer retry had a wide spread. Thus weighted cost is an *assumption-sensitive proxy*, not a measured runtime prediction on this machine. Starting-guess computation, division, roots and other arithmetic are not fully included.

A FUSED PAIR DOES NOT ESTABLISH A COMPILED SPEED-UP

The Python implementation obtains sin and cos with two separate calls. For the representative median solves, let p be the cost of a pair in units of one sin call. NWM11 costs $4 + p$ and Danby $3 + 2p$: at the configured $p = 2.34$ the model gives 6.34 versus 7.68, but at $p = 1$ both cost 5. Even at $p = 1.1$ the gap is only about 2%. Across converged solves on the *whole grid*, NWM11 with its canonical guess has a mean modelled cost of 9.55 versus Danby with Napier’s 7.48; the call-count proxy gives 6.30 versus 4.88. A compiled implementation and a direct timing comparison would be needed before claiming a speed advantage.

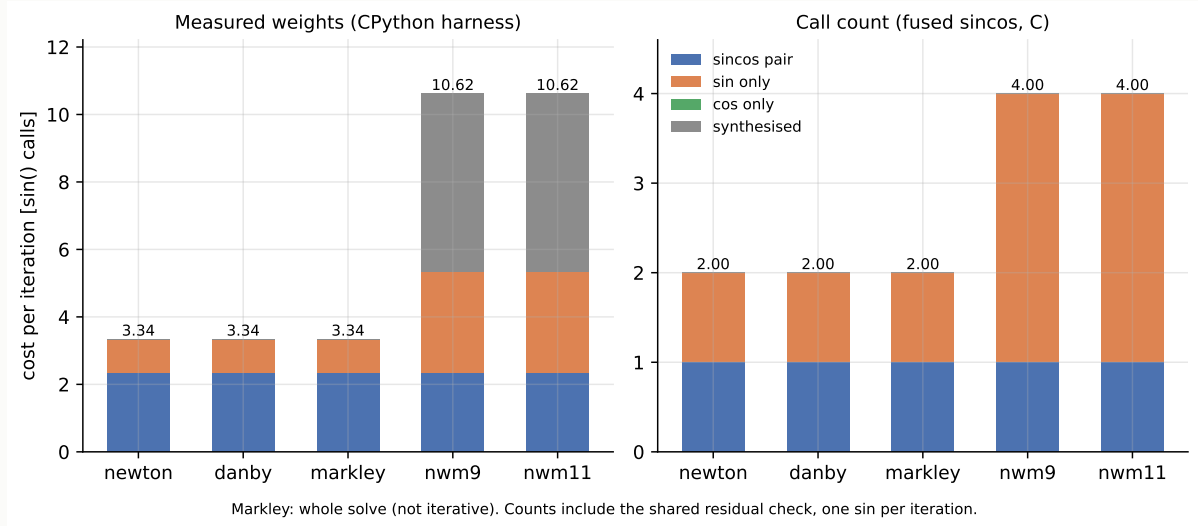


Figure 1: Steady-state per-iteration proxy under configured Python weights (left: 3.34 for Newton and Danby, 10.62 for NWM9 and NWM11) and one-call-per-evaluation-point counting (right: 2 and 4). This is *per iteration*, not per completed solve; the latter also depends on the number of iterations and the starting guess. Markley’s non-trigonometric arithmetic is not represented. Both panels include one residual check.

5 GRID BENCHMARK

Every solver-guess combination was run over a 7,400-point (e, M) grid combining a uniform block, a logarithmically-spaced pathological corner and a RadVel-realistic sample: 17 combinations $\times$ 7,400 points = **125,800 instrumented solves**. Markley ignores the starting guess and is run once under the label n/a.

5.1 HEADLINE COMPARISON

Each iterative solver is shown with a start that gave strong convergence on this grid. Mean iterations and modelled cost use converged grid solves; the wall-clock column uses medians over 50 timed points, each repeated 1000 times. The two cost columns answer different questions and should not be read as interchangeable.

Solver	Guess	Mean iter.	Median cost	Median wall (μ s)	Converged
Markley	n/a	0.00	3.34	3.50	100.00%
NWM11	canonical	1.38	6.34	5.25	99.93%
NWM9	napier	1.44	6.34	6.52	100.00%
Danby	napier	1.94	7.68	4.46	100.00%
Newton	napier	3.21	11.02	4.40	100.00%

The median does not describe the costly tail.

Iteration count. NWM11 needs a median of one iteration where Danby needs two and Newton needs three. This is consistent with the measured convergence orders, but most one-iteration solves cannot benefit from recycled information from a previous iteration.

Weighted cost. The chosen NWM11 median is 6.34 against Danby's 7.68, about 17% lower under the configured weights. Their means reverse the ordering: 9.55 against 7.48. Under one-call-per-point counting the means are 6.30 against 4.88. The median alone therefore does not establish an overall cost advantage.

Wall-clock time. On this Python implementation and timed subset, NWM11 and NWM9 with the displayed guesses take about 1.2 and 1.5 times Danby's median time. A weak simple start makes NWM9 much slower (17.69 μ s); runtime is not fixed by the solver name alone.

5.2 THE GUESS LAYER

The starting guess can change both iteration count and reliability. Newton's mean count on converged solves moves from 3.84 with $E_0 = M$ to 3.21 with Napier's formula, and convergence rises from 99.88% to 100%. NWM9 changes from 99.43% with the simple start to 100% with Napier's. The guess cost itself is excluded from the modelled cost counters but included in wall-clock timing.

This is why the guess is treated as an independent experimental factor.

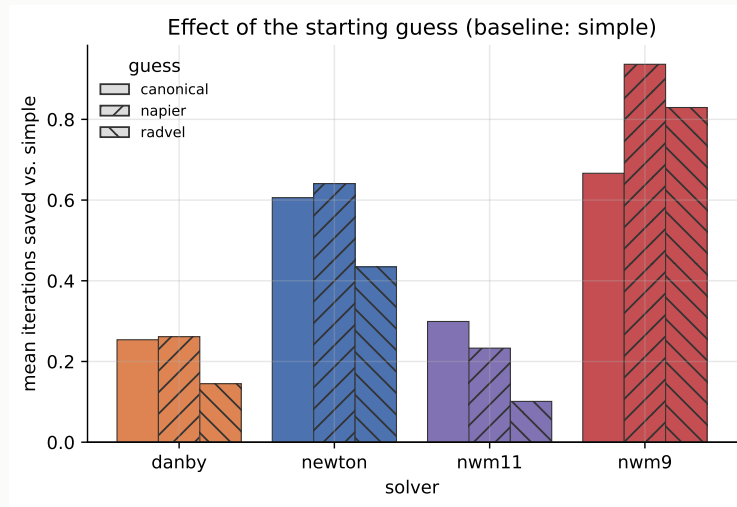


Figure 2: Mean iterations saved against $E_0 = M$ on the full grid. Napier's guess has the largest improvement for Newton and NWM9; NWM11 instead has its lowest mean count with the canonical guess.

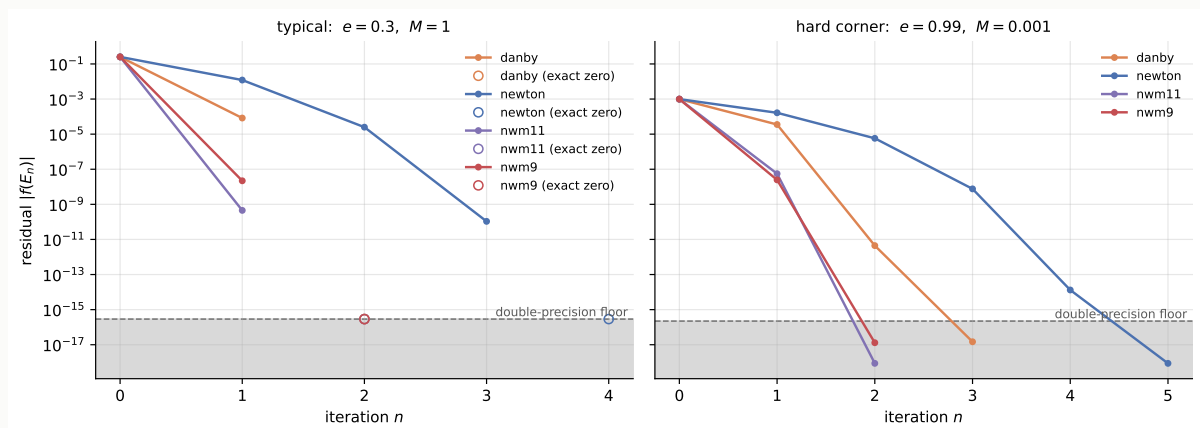


Figure 3: Residual histories at a typical (e, M) and in the pathological corner. Both with-memory schemes reach the double-precision floor in two iterations against Newton's four to five. The shaded band marks the floor below which a curve measures rounding rather than convergence.

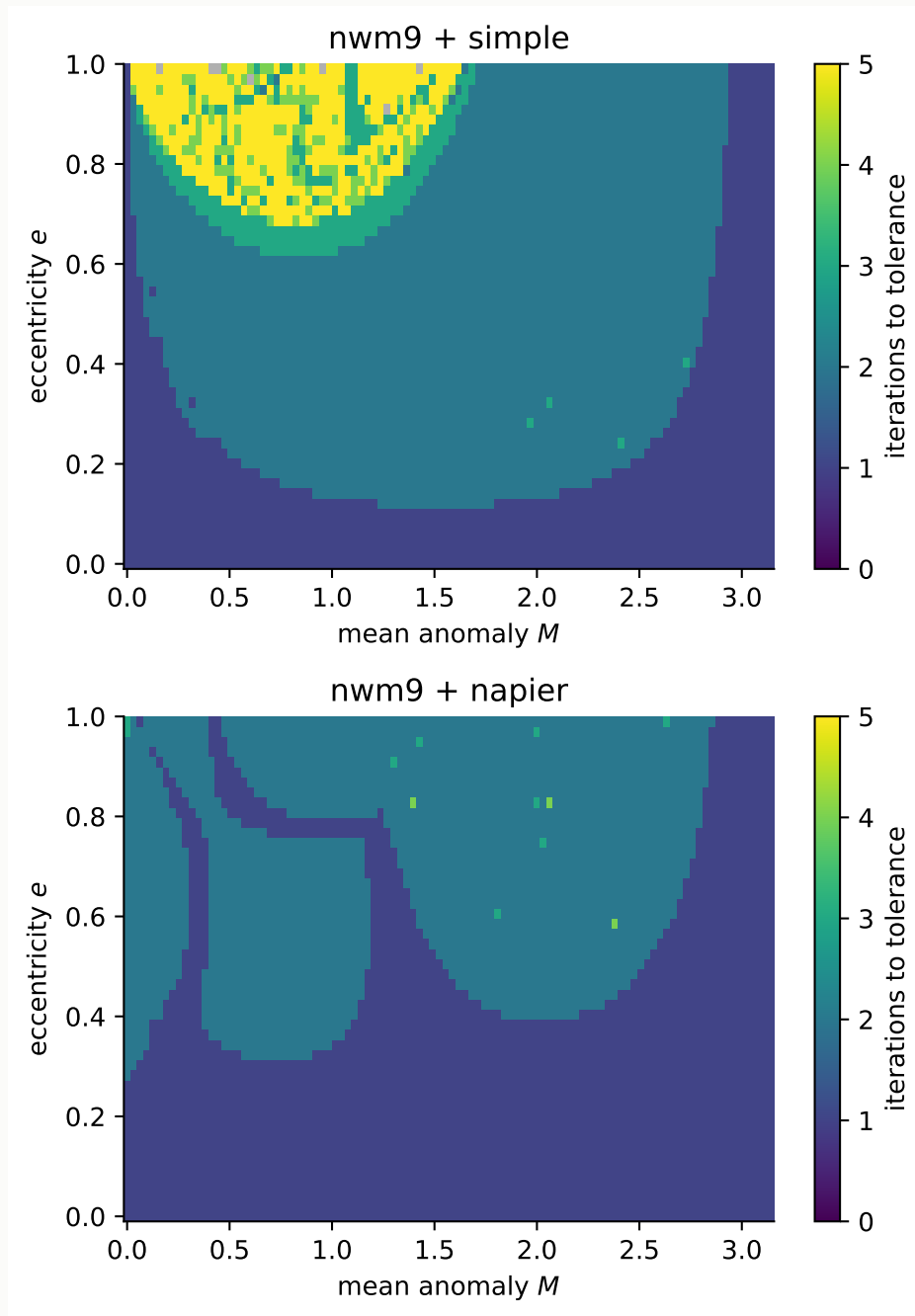


Figure 4: NWM9 iterations with the simple and Napier starts on the complete uniform 50×100 grid ($0 \leq e \leq 0.99$, $0 \leq M \leq \pi$), on a shared colour scale. The simple start has a much larger high-iteration region; the logarithmically sampled extreme corner is shown separately in the failure maps.

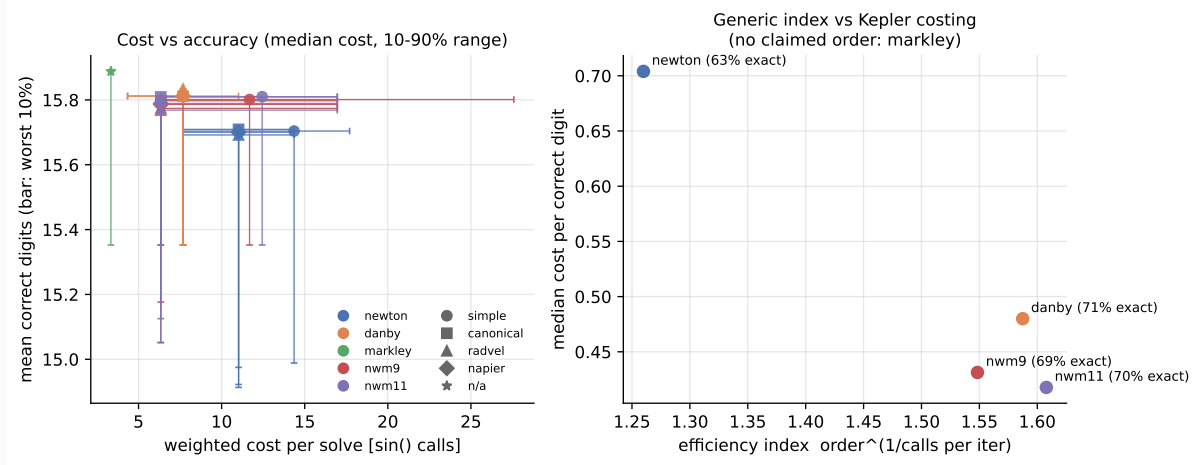


Figure 5: *Left*: attained accuracy against modelled cost per solve; successful methods commonly reach the double-precision floor. *Right*: the generic order-per-call index against median cost per correct digit. Newton does not lead on this measure: its median is 0.70, versus 0.42 for NWM11 and 0.48 for Danby. The two rankings are broadly similar; this experiment does not show the large rank reversal anticipated in the proposal. The digit calculation includes exact-reference hits at the project’s 16-digit ceiling.

6 ROBUSTNESS

Failure rates split by region. The current analysis code defines the hard corner as $e > 0.9$ and $M < 0.1$; boundary samples at exactly 0.9 or 0.1 are classified as ordinary. Counts below are *solves* across four guesses for each iterative method, not distinct grid points.

Solver	Corner solves	Corner failure	Ordinary solves	Ordinary failure
Markley	381	0.000%	7,019	0.000%
Danby	1,524	0.131%	28,076	0.000%
Newton	1,524	0.919%	28,076	0.018%
NWM9	1,524	4.265%	28,076	0.057%
NWM11	1,524	4.856%	28,076	0.000%

ROBUSTNESS IS A SOLVER–GUESS RESULT

NWM11 has no failures in the ordinary region, but 74 failed solves in the hard corner across its four guesses. Its canonical guess fails on 5 of 381 corner points; its RadVel guess fails on 35. NWM9 with Napier’s guess has no failures anywhere on this grid, despite 81 failed solves for NWM9 across all guesses.

The small derivative $f'(E) = 1 - e \cos E$ makes this corner difficult. Freezing the accelerating parameters can improve some failures, but that changes both their initial values and the recycling rule. The current experiment does not isolate memory recycling as the sole cause of failure; the starting guess is clearly important.

6.1 ORDER ACROSS THE ECCENTRICITY RANGE

Convergence order measured on Kepler’s equation itself at $M = 0.3$, at 2000-digit precision — a different and more demanding question than the generic test functions of Section 3.

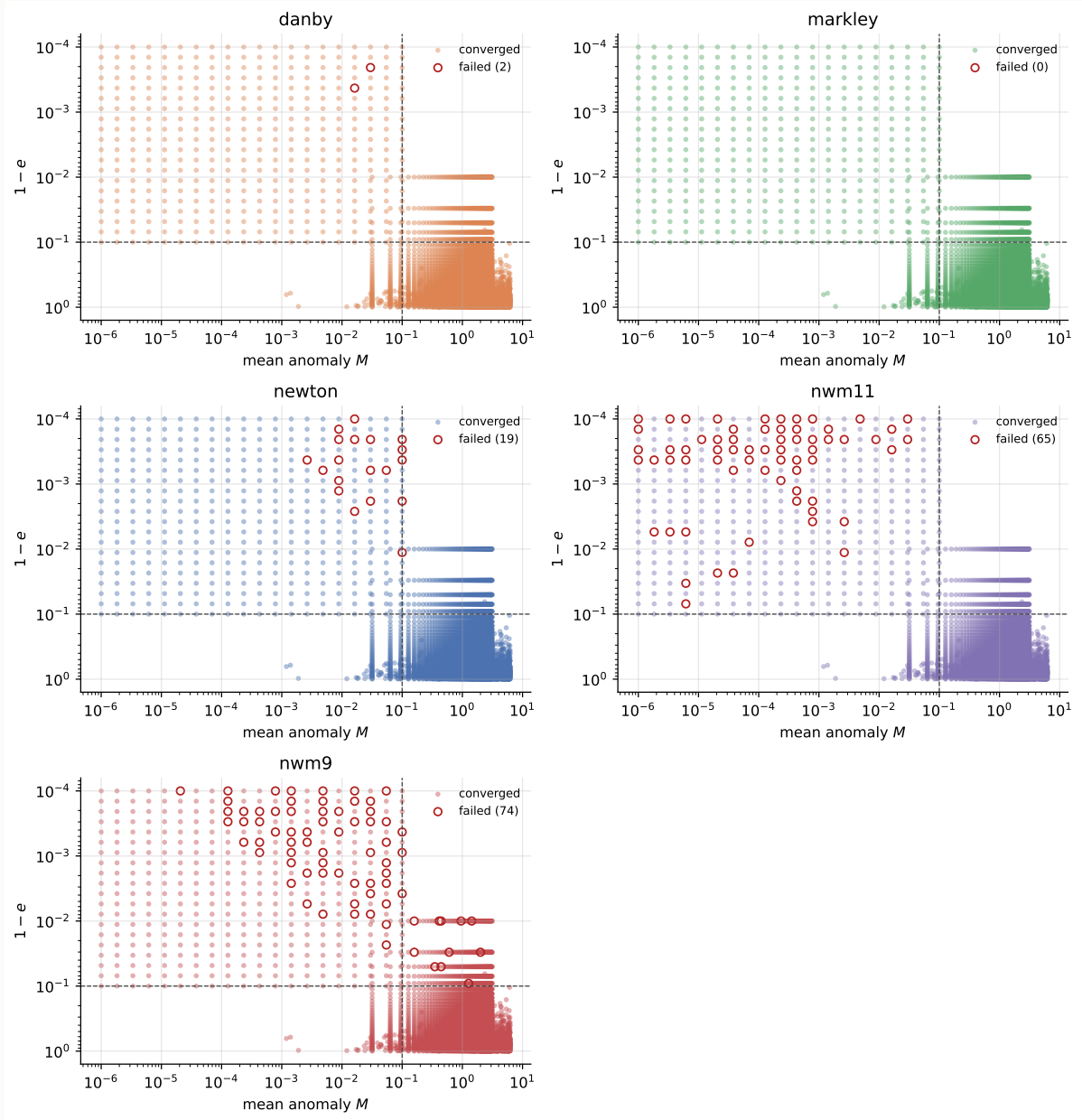


Figure 6: Where each solver fails, on logarithmic axes in $1 - e$ and M so the pathological corner occupies the top left rather than a sliver. The map marks a *distinct point* if any guess fails there: Markley 0, Danby 2, Newton 19, NWM11 65, NWM9 74. These need not equal the failed-solve counts in the table. Points at exactly $e = 0$ or $M = 0$ cannot be drawn on log axes.

e	0.1	0.3	0.5	0.7	0.9	0.95	0.99	0.995	0.999
Newton	2.000	2.000	2.000	2.000	2.000	2.000	2.129	—	1.786
Danby	4.000	4.000	4.000	4.000	4.000	4.000	4.000	4.000	4.000
NWM9	8.891	8.897	8.910	8.896	2.820	8.899	4.479	0.721	0.365
NWM11	11.233	10.716	10.728	10.756	10.876	10.839	10.733	10.739	10.744

Along this $M = 0.3$ slice, NWM11's measured order remains near its claimed 10.7446 up to $e = 0.999$; Danby's is 4.000. This test does not establish the same behaviour as $M \rightarrow 0$.

NWM9's estimated order drops beyond $e = 0.95$ in this slice. However, each extreme estimate uses only a few pre-floor errors, the dip at $e = 0.9$ recovers immediately, and no independent implementation of NWM9 has been checked here. These are observations about the estimator and implementation, not proof that the mathematical scheme changes order. Newton's missing or low extreme- e estimates have the same resolution caveat.

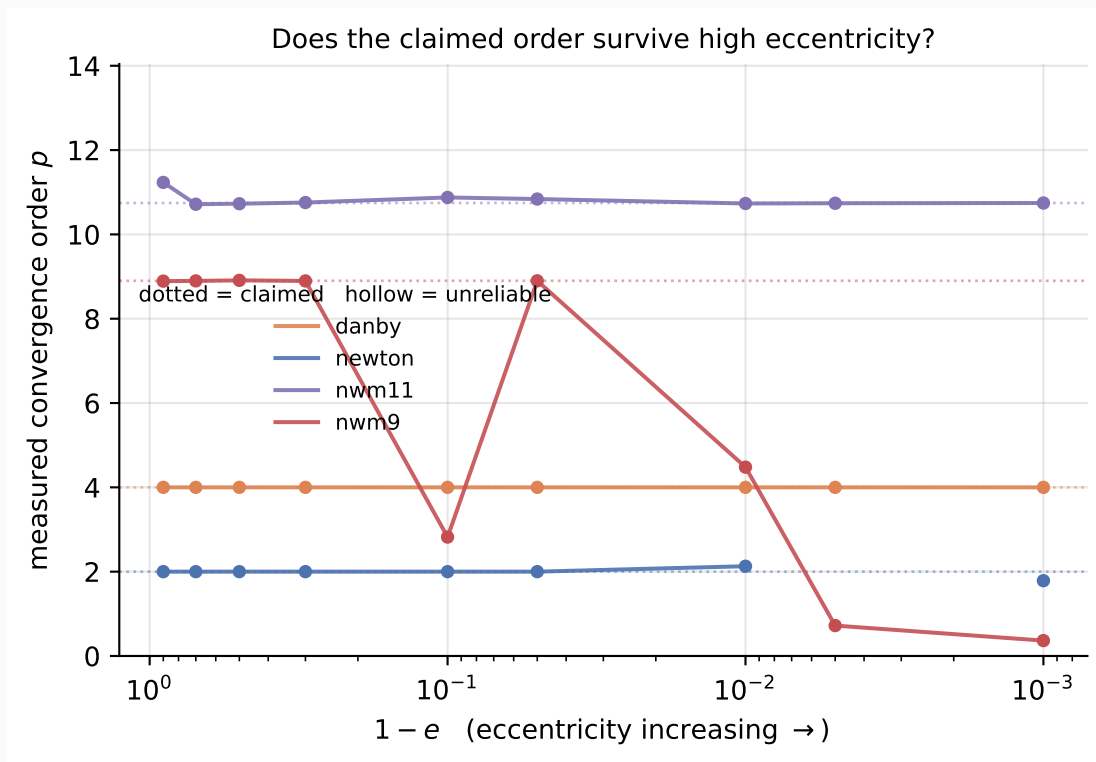


Figure 7: Measured order at $M = 0.3$. Dotted lines mark published orders; hollow markers identify estimates with limited resolution. NWM9's extreme- e estimates need independent verification.

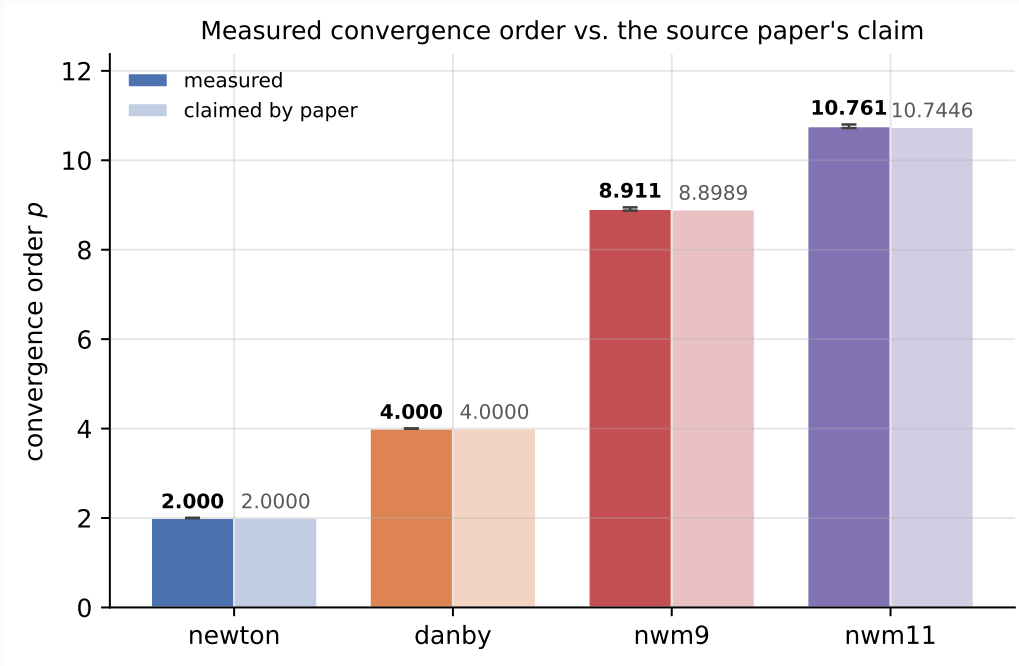


Figure 8: Measured convergence order against each source paper’s claim, on the NWM11 paper’s eight test functions (used for every solver). Error bars show the spread across the eight functions.

7 DOWNSTREAM: DOES ANY OF IT MATTER?

Solver error was traced through $M \rightarrow E \rightarrow v \rightarrow v_r$ into refitted orbital parameters, with each of our solvers patched into RadVel in place of its own compiled solver. A call counter asserts after every fit that the patch actually fired.

Design: injection-recovery on real cadence. The observation times, the per-point uncertainties and the three-instrument structure come from 401 HD 164922 measurements. The velocities are *not*: they are replaced by a known, moderately eccentric orbit ($P = 20.885258$ d, $e = 0.35$, $\omega = 1.0$ rad, $K = 5 \text{ m s}^{-1}$) plus Gaussian noise at the real uncertainties. Fitting HD 164922’s own velocities with a single-planet model would give a near-circular, optimiser-dependent eccentricity in a one-planet fit (the system has four planets). The injection instead provides a known answer to recover. The fit varies P , the time of conjunction, $\sqrt{e} \cos \omega$, $\sqrt{e} \sin \omega$ and $\log K$ (RadVel’s recommended basis), plus one jitter term per instrument. Before the final fit, a 30-point multi-start over (e, ω) using RadVel’s native solver picks the starting point. The fit recovers $e \approx 0.342$, $\omega \approx 0.997$ and $K \approx 5.03$, close to the injected values for all five solvers. Each fit performs about 1.3×10^5 Kepler solves.

7.1 COST OF A FULL FIT

Solver	Median fit (s)	Slowdown vs. native	µs per solve
Newton	0.95	17.3×	7.12
Danby	1.03	18.5×	7.35
Markley	1.00	22.3×	7.68
NWM11	1.23	27.8×	9.24
NWM9	1.35	30.6×	9.69

Medians over the six tolerances (results/error_propagation/timing.csv). Only the fit

made with the solver under test is timed, not the multi-start search. “Native” is the identical fit, from the same starting point, using RadVel’s compiled solver (about 0.04–0.05 s in this run). Each fit was timed once, so small differences are not reliable rankings.

The slowdown against native RadVel includes Python-to-scalar conversion: our solvers are scalar and RadVel passes arrays. This does not isolate algorithmic speed from implementation overhead. On this setup, the with-memory fits take longer than the classical ones.

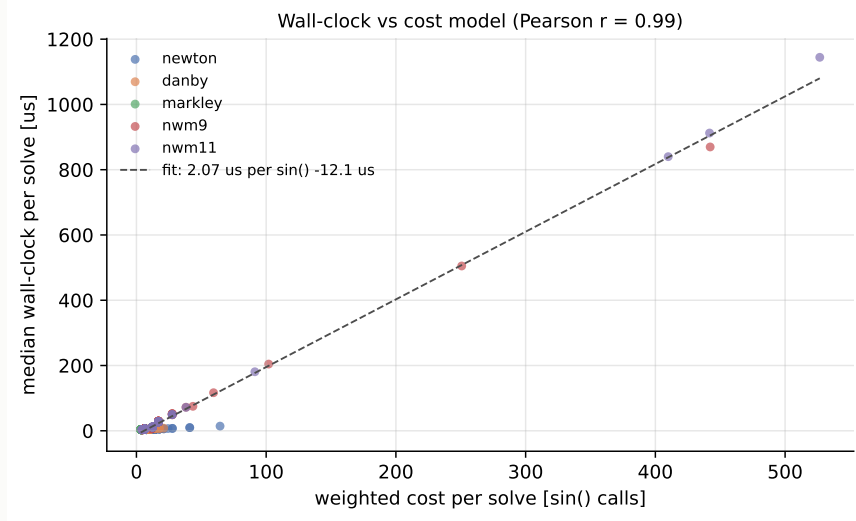


Figure 9: One point per timed (e, M) solve and solver–guess pair (850 points). Pearson $r \approx 0.99$ is dominated by five very slow corner solves; without them it falls to 0.83, and the full-sample rank correlation is 0.49. The proxy does not reliably rank ordinary solves by runtime.

7.2 THE ERROR BUDGET

The question is not how large the solver-induced parameter shift is, but how it compares to the two uncertainties that are already present.

Parameter	Solver shift (worst case)	Noise spread (Monte Carlo)	MCMC posterior (1σ)	Noise / solver
P (d)	1.0×10^{-8}	5.04×10^{-4}	6.44×10^{-4}	5.1×10^4
e	1.1×10^{-5}	1.65×10^{-2}	1.75×10^{-2}	1.5×10^3
ω (rad)	2.3×10^{-5}	5.03×10^{-2}	5.59×10^{-2}	2.2×10^3
K (m s^{-1})	3.5×10^{-5}	9.93×10^{-2}	1.04×10^{-1}	2.8×10^3

The solver shift is the largest absolute shift over all five solvers and all tolerances, measured from each solver’s own 10^{-14} fit. In every parameter that worst case is Newton at 10^{-4} . The noise spread is the standard deviation over 50 refits of fresh noise realisations. The MCMC width comes from one RadVel run with the native solver, configured with `nrun=2000`; RadVel’s convergence checks may extend the total run.

NUMERICAL PRECISION IS NOT LIMITING IN THIS INJECTION

For this $e = 0.35$ injected orbit, even the loosest tested tolerance (10^{-4}) gives parameter shifts at least about 10^3 times smaller than the spread across 50 noisy refits. Newton at 10^{-4} has the largest observed effect, $6.3 \times 10^{-4}\sigma$ in e . At other settings the shifts are at most about $1.1 \times 10^{-5}\sigma$; their non-monotonic variation is compatible with fit/optimiser

resolution and is best treated as an upper bound on solver effects. Markley ignores the tolerance, so its six fits are identical; the 10^{-14} row is zero by construction for every solver.

The Monte Carlo spread and MCMC posterior width have similar scales in this example, but they are different distributions: repeated noisy refits measure estimator sampling variation, whereas MCMC conditions on the observed data and priors. Agreement is a check of scale, not an expectation that the two distributions coincide.

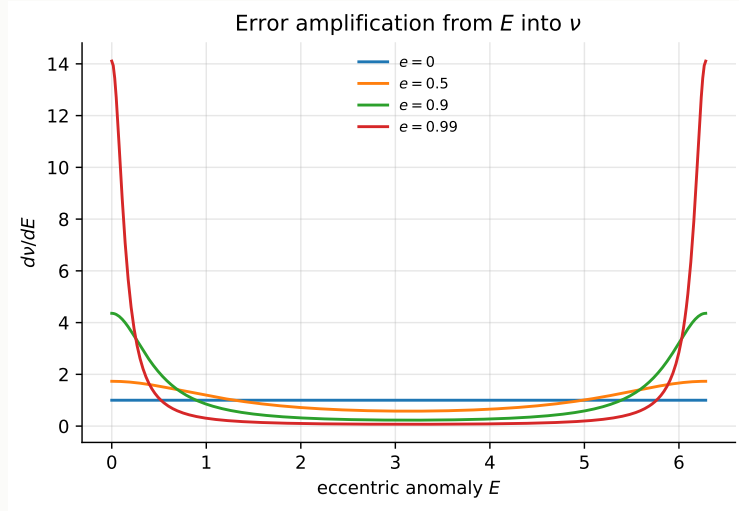


Figure 10: The amplification dv/dE carrying solver error from the eccentric anomaly into the true anomaly. Near-circular orbits pass error through roughly unchanged; at $e = 0.99$ the factor exceeds 14 at the ends of the range. This illustrates a possible sensitivity; the downstream fit experiment used $e = 0.35$, not $e = 0.99$.

7.3 REAL-DATA CHECKS

The main study uses injected velocities, so we also fitted all five solvers to three datasets' *real*, unmodified velocities at one tight tolerance. This separate check tests agreement between solvers, not recovery of a known orbit. For the two transiting planets, period and transit time are held fixed from external measurements:

- K2-24 b: $P = 20.885258$ d, $t_c = 2072.79438$.
- K2-131 b: $P = 0.3693038$ d, $t_c = 2457582.9360$ [9].

HD 164922 b does not transit, so only its period (1207 d) is fixed. The fits use the same basis and starting-point search for every solver.

Dataset	Points	Fitted e (approximately)
K2-131 (1 planet)	70	0.135
HD 164922 (4 planets)	401	0.122
K2-24 (2 planets)	32	0.908

The five solver outputs agree to the displayed precision. HD 164922 and K2-24 are multi-planet systems fitted here with one planet, so the reported eccentricities should not be interpreted as new measurements of their orbits. These direct checks were rerun separately from

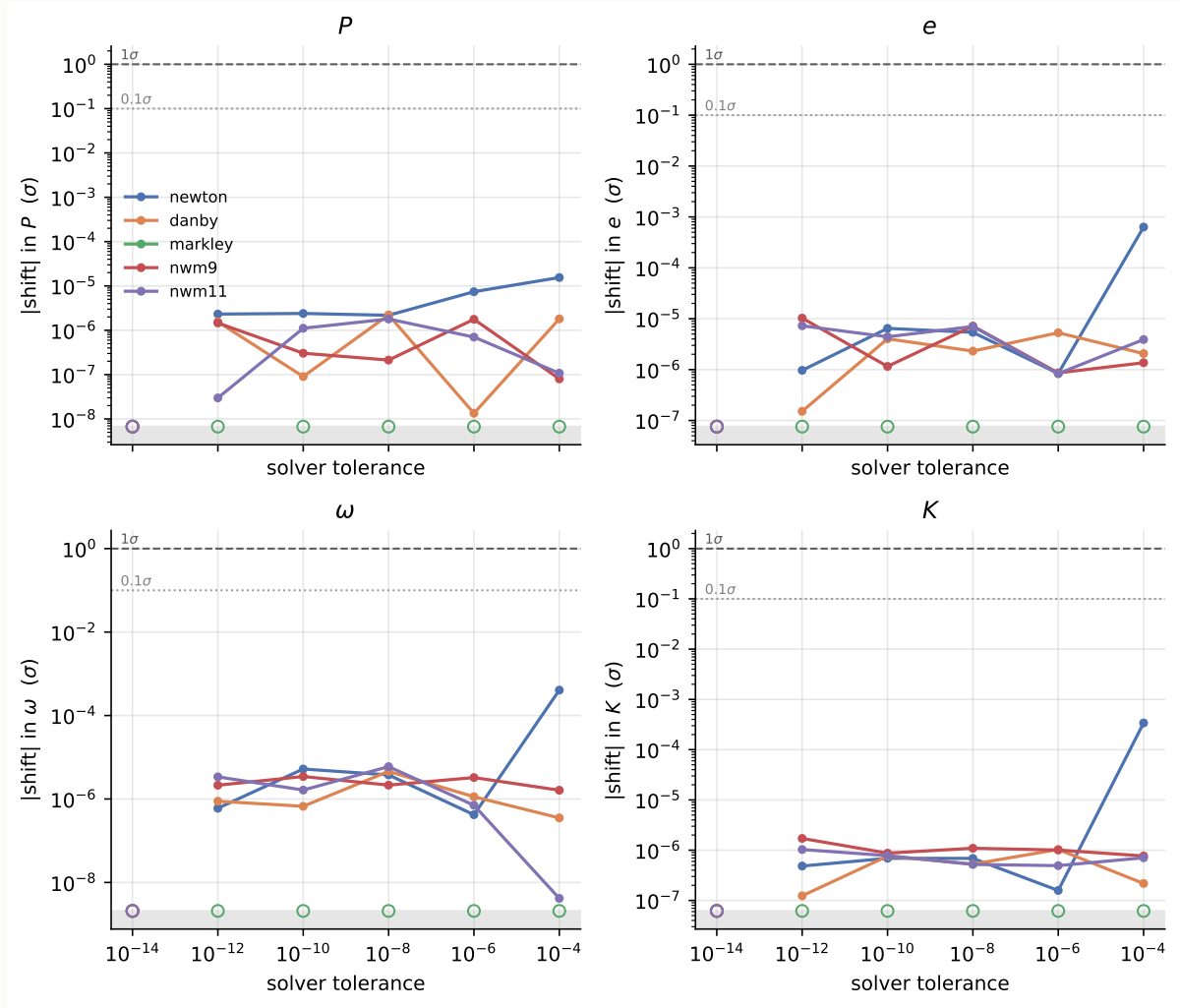


Figure 11: Shift in each fitted parameter as the solver tolerance is loosened, in units of the MCMC posterior σ . The dotted guides mark 1σ and 0.1σ . All plotted shifts are below $10^{-3}\sigma$ for this injected orbit. Hollow markers denote zero shifts: Markley at every tolerance and every solver at the 10^{-14} reference.

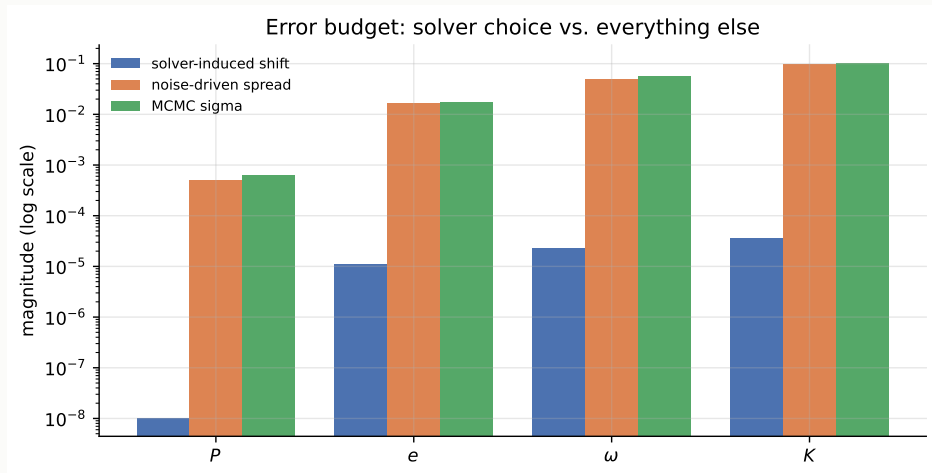


Figure 12: Solver-induced shift, repeated-noise spread and MCMC posterior width for the injected orbit. The latter two have similar scales but different statistical meanings; the maximum solver shift is at least three orders of magnitude smaller than the noise spread.

the configured injection experiment; their 15 fit rows are saved with the experiment evidence as `real_data_checks.csv`.

8 SYNTHESIS

The results separate formal convergence, completed-solve cost and the effect on a fitted orbit:

Axis	Verdict
Convergence order	On the generic verification functions, NWM11 measures 10.76 versus Danby's 4.00. Its order on Kepler was also near the published value along the tested $M = 0.3$ slice.
Iteration count	Selected NWM11 and NWM9 starts have a median of one iteration versus Danby's two.
Modelled cost	The selected NWM11 median is 6.34 versus 7.68 for Danby, but the corresponding converged-solve means are 9.55 versus 7.48; the weights are machine-sensitive.
Python wall-clock	Timed-grid medians for those starts are 5.25 and 4.46 μ s per solve. Full-fit medians also favour the simpler implementations on this setup.
Robustness	Failures concentrate near $e \rightarrow 1$, $M \rightarrow 0$ and depend strongly on the guess. NWM9 with Napier has none on this grid.
Downstream fit	For one injected $e = 0.35$ orbit, the largest parameter shifts are at least about 10^3 below the repeated-noise spread.

The proposal's numerical-cost hypothesis is therefore only partly supported: fewer iterations and a lower *selected median* proxy do not translate into a lower average cost or a Python runtime win. A fused pair would erase the representative median proxy gap at $p = 1$; compiled speed remains an experiment to perform, not a conclusion of this one.

Markley has the lowest trigonometric proxy (3.34) and fastest median on the timed grid subset, but the proxy omits some of its arithmetic and its closed-form interface does not enforce the iterative tolerance. Six iterative solver-guess pairs also have zero failures on this grid. Danby remains the appropriate production reference among the iterative methods studied here.

9 THREATS TO VALIDITY

1. **Implementation and calibration.** Wall-clock timing compares scalar Python implementations, not compiled kernels. The cost proxy uses weights calibrated under Python 3.14.7, whereas this rerun used Python 3.12.6; local recalibration was unstable. It also omits starting-guess cost and some non-trigonometric work.
2. **The residual-evaluation artefact.** The shared loop's own $f(E)$ call inflates every solver's sin-only count by one per iteration. It is uniform across methods and does not affect some rankings, but the absolute per-iteration figures in Section 4 are one sin high. Markley is not subject to the same tolerance stopping test as the iterative solvers.
3. **One injected orbit.** The error-propagation and Monte Carlo studies use one injected orbit ($e = 0.35$) on HD 164922's real cadence and uncertainties, not its own velocities. High- e injections were not run; the real-data checks only establish agreement among solvers under the selected fit model.
4. **Optimiser resolution floor.** Below tolerance 10^{-4} the measured shifts (outside Newton at 10^{-4} , at most about $1.1 \times 10^{-5}\sigma$) are compatible with optimiser noise; they bound rather than directly measure solver effects.
5. **Uncertainty estimates.** The Monte Carlo sample has only 50 realisations. The MCMC width comes from one RadVel run configured with `nrun=2000`; RadVel may extend the total run to satisfy mixing checks. Sampling spread and posterior width need not match.

6. **Region and provenance.** The hard-corner classifier excludes its boundary samples. Several derived/downstream CSVs lack provenance sidecars, although the benchmark raw data has one.
7. **Estimator artefacts at extreme eccentricity.** Several entries in the order-versus-eccentricity table are measurement artefacts rather than method behaviour, as noted in Section 6. They are reported rather than removed, with the unreliable points marked.
8. **NWM9's degradation is single-sourced.** The collapse beyond $e \approx 0.99$ has been reproduced across precisions but not cross-checked against an independent implementation of the same published scheme.

10 CONCLUSION

Five Kepler solvers were checked through one instrumented harness over 125,800 solves. The iterative implementations passed the project's order and correctness gates on the selected verification sets, but NWM9 still needs a check on its own paper's examples and an independent implementation at extreme eccentricity.

The newer with-memory schemes need fewer iterations with suitable starts. Their selected median proxy cost is lower than Danby's, while their average proxy cost and measured Python runtime are higher. Reliability in the hard corner depends strongly on the starting guess; it cannot be attributed to memory alone from the present evidence.

In the one injected-orbit fit studied here, solver-tolerance changes are small compared with noise-driven parameter spread. Danby is a sound iterative production baseline for this implementation. Whether a compiled version of either newer method is faster remains untested.

REFERENCES

- [1] B. J. Fulton, E. A. Petigura, S. Blunt, and E. Sinukoff, “RadVel: The Radial Velocity Modeling Toolkit,” *Publications of the Astronomical Society of the Pacific*, vol. 130, no. 986, p. 044504, 2018. doi: [10.1088/1538-3873/aaaaa8](https://doi.org/10.1088/1538-3873/aaaaa8).
- [2] J. M. A. Danby, “The solution of Kepler’s equation, III,” *Celestial Mechanics*, vol. 40, nos. 3–4, pp. 303–312, 1987. doi: [10.1007/BF01235847](https://doi.org/10.1007/BF01235847).
- [3] F. L. Markley, “Kepler equation solver,” *Celestial Mechanics and Dynamical Astronomy*, vol. 63, no. 1, pp. 101–111, 1995. doi: [10.1007/BF00691917](https://doi.org/10.1007/BF00691917).
- [4] S. K. Mittal, S. Panday, and L. Jäntschi, “Enhanced Ninth-Order Memory-Based Iterative Technique for Efficiently Solving Nonlinear Equations,” *Mathematics*, vol. 12, no. 22, p. 3490, 2024. doi: [10.3390/math12223490](https://doi.org/10.3390/math12223490).
- [5] S. K. Mittal, S. Panday, L. Jäntschi, and L. C. Bolunduğ, “Two novel efficient memory-based multi-point iterative methods for solving nonlinear equations,” *AIMS Mathematics*, vol. 10, no. 3, pp. 5421–5443, 2025. doi: [10.3934/math.2025250](https://doi.org/10.3934/math.2025250).
- [6] K. J. Napier, “Improved Initial Guesses for Numerical Solutions of Kepler’s Equation,” *arXiv preprint arXiv:2411.15374*, 2024. doi: [10.48550/arXiv.2411.15374](https://doi.org/10.48550/arXiv.2411.15374).
- [7] E. A. Petigura *et al.*, “Two Transiting Low Density Sub-Saturns from K2,” *The Astrophysical Journal*, vol. 818, no. 1, p. 36, 2016. doi: [10.3847/0004-637X/818/1/36](https://doi.org/10.3847/0004-637X/818/1/36). (K2-24 system; see `data/README.md` for the full dataset provenance.)
- [8] B. J. Fulton *et al.*, “Three Temperate Neptunes Orbiting Nearby Stars,” *The Astrophysical Journal*, vol. 830, no. 1, p. 46, 2016. doi: [10.3847/0004-637X/830/1/46](https://doi.org/10.3847/0004-637X/830/1/46). (HD 164922 system.)
- [9] F. Dai *et al.*, “The Discovery and Mass Measurement of a New Ultra-short-period Planet: K2-131b,” *The Astronomical Journal*, vol. 154, no. 6, p. 226, 2017. doi: [10.3847/1538-3881/aa9065](https://doi.org/10.3847/1538-3881/aa9065). (K2-131 ephemeris used in the real-data checks.)